

Bicycle Simulator Documentation

Mixed Reality Lab Project, SS24

Pooja Ramesh, Philipp Baumann, Philipp Senfft, Marc Hessenauer

Table of Contents

1.	<i>Introduction</i>	2
2.	<i>Project Motivation and Goals</i>	2
3.	<i>Physical Bicycle Setup</i>	3
4.	<i>Environment Setup in Unity</i>	6
5.	<i>Use Case Analysis and Scenario Derivation</i>	9
6.	<i>Project Structure in Unity</i>	12
7.	<i>Scenario Implementation</i>	13
	Parameter 1: Lighting	13
	Parameter 2: Other Road Users	13
	Parameter 3: Traffic Regulation	16
8.	<i>Integration with Elite Rizer</i>	20
	Reverse Engineering Rizer	20
9.	<i>Summary and Next Steps</i>	23
10.	<i>Sources</i>	24

1. Introduction

In today's technological world, the need for environmentally friendly transportation is increasingly important. One effective and simple mode of transportation, particularly in German urban areas, is the bicycle, whether traditional or electric. However, many people avoid cycling due to anxiety about busy and potentially dangerous traffic. A fear of bicycles is a known occurrence described as Cyclophobia in the Psych Times [1]. To address this, this project proposes a simulated environment that identifies anxiety-inducing factors without posing any real-life threats. This risk-free method will help us better understand and mitigate the concerns of potential cyclists.

2. Project Motivation and Goals

The primary motivation behind this project was to address these concerns and encourage more people to cycle. By creating a simulated environment, we aimed to identify and understand the factors that make cycling challenging or intimidating. This approach allowed us to analyze these issues and develop strategies to improve the cycling experience, making it safer and more appealing for everyone.

The specific goals of this project were as follows:

- **Identifying Barriers to Cycling:** Develop a virtual simulation to pinpoint specific situations and elements in urban traffic that cause anxiety among cyclists, such as busy intersections, narrow spots, and overtaking vehicles.
- **Improving Driving Practice:** Use the simulation to create scenarios that expand cyclists' driving practice, focusing on intersections, overtaking, narrow spots, and different times of the day.
- **Enhancing Cyclist Safety:** Test and evaluate scenarios that help cyclists build confidence and feel safe on the road, especially when sharing the road with cars, trucks, and buses.
- **Weather and Environmental Factors:** Simulate varying weather conditions and times of day to assess their impact on cyclists' stress and anxiety levels and help them adapt to these conditions.
- **Building Confidence in Different Scenarios:** Develop specific scenarios that allow cyclists to practice and become comfortable with situations like riding next to the road, overtaking cars, cycling at night, and dealing with obstacles and dangerous situations.

3. Physical Bicycle Setup

The bicycle simulator is designed to offer an immersive and realistic cycling experience by integrating both commercial and custom-built hardware components.

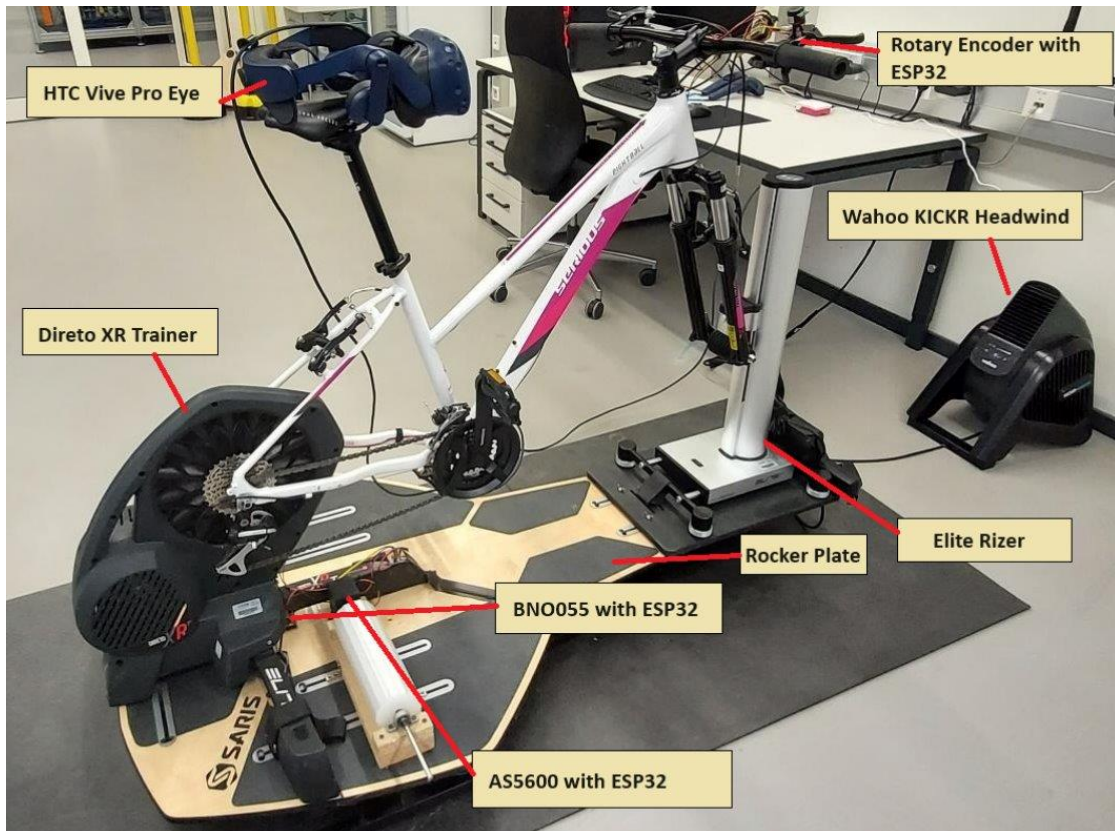


Figure 1: Physical Bicycle Setup

This section provides an overview of the physical setup of the simulator, including the devices used, their placements, and their functions.

Commercial Devices

1. Direto XR

- a. **Functionality:** The Direto XR is a smart trainer placed on the back axle of the bicycle. It provides real-time feedback on the speed generated by the cyclist and allows us to control the resistance, simulating different terrains and inclines.
- b. **Communication:** Bluetooth Low Energy (BLE).
- c. **Data Interaction:**
 - i. **Read:** Speed generated by the cyclist.
 - ii. **Write:** Resistance to simulate different riding conditions.

2. Elite RIZER

- a. **Functionality:** The Elite RIZER is mounted on the front axle and adjusts the bike's incline to simulate uphill and downhill riding. It also reads the steering inputs from the cyclist.
- b. **Communication:** Bluetooth Low Energy (BLE).
- c. **Data Interaction:**
 - i. **Read:** Steering (65 for right, 193 for left, and other values for straight out). So, we get just a binary steering form the RIZER.
 - ii. **Write:** Incline of the bike.

3. KICKR Headwind

- a. **Functionality:** This device simulates wind resistance, providing a more immersive riding experience by varying airflow based on the cyclist's speed.
- b. **Communication:** Bluetooth Low Energy (BLE).

Custom-Built Devices

1. Brake System

- a. **Components:** Rotary Encoder, controlled by an ESP32 SoC.
- b. **Functionality:** The brake system simulates the effect of braking to stop the bike.
- c. **Communication:** Data is sent over UDP to the main computer via the ESP32's built-in Wi-Fi.

2. Reverse Drive Mechanism

- a. **Components:** AS5600 Hall Effect Sensor, controlled by an ESP32 SoC.
- b. **Functionality:** This mechanism assists the cyclist in reversing when they get stuck, similar to pushing off from a stop.
- c. **Communication:** Data is sent over UDP to the main computer via the ESP32's built-in Wi-Fi.

3. Lean Sensor

- a. **Components:** BNO055 IMU, controlled by an ESP32 SoC.
- b. **Functionality:** The lean sensor detects the cyclist's body lean during turns, enhancing the realism of steering and balance.
- c. **Communication:** Data is sent over UDP to the main computer via the ESP32's built-in Wi-Fi.

Integration and Communication

All devices, except for the KICKR Headwind, are mounted on a rocker plate, which allows for realistic lateral movement and balance adjustments during the ride. The system's architecture also includes a gaming PC equipped with HTC Vive Pro VR glasses, providing an immersive virtual reality environment.

To streamline communication and control, we use a Raspberry Pi as an interface between the hardware components and the gaming PC running Unity. However, for development purposes, we have configured the gaming PC to communicate directly with all devices, bypassing the Raspberry Pi. This setup simplifies the development and debugging processes.

Development Insights

When our team started the project, the previous groups had already implemented the basic operations of all devices. The Direto XR had the speed-reading function implemented, and its API for writing resistance was available but not yet utilized. Our main task was to reverse engineer the Elite RIZER, which had the steering read function implemented, but the API for writing the incline needed to be developed from scratch.

By integrating these commercial and custom-built devices, our bicycle simulator provides a comprehensive and realistic cycling experience, simulating various physical conditions and rider inputs in real-time.

4. Environment Setup in Unity

An environment containing different elements was required to implement the use cases described in chapter x. Many use cases take place at intersections, some with and some without traffic lights. Hence, the environment has several intersections. Since most intersections are found in cities, the environment was created as a city scene.

This is a map of what the environment looks like without being adapted to a specific scenario:

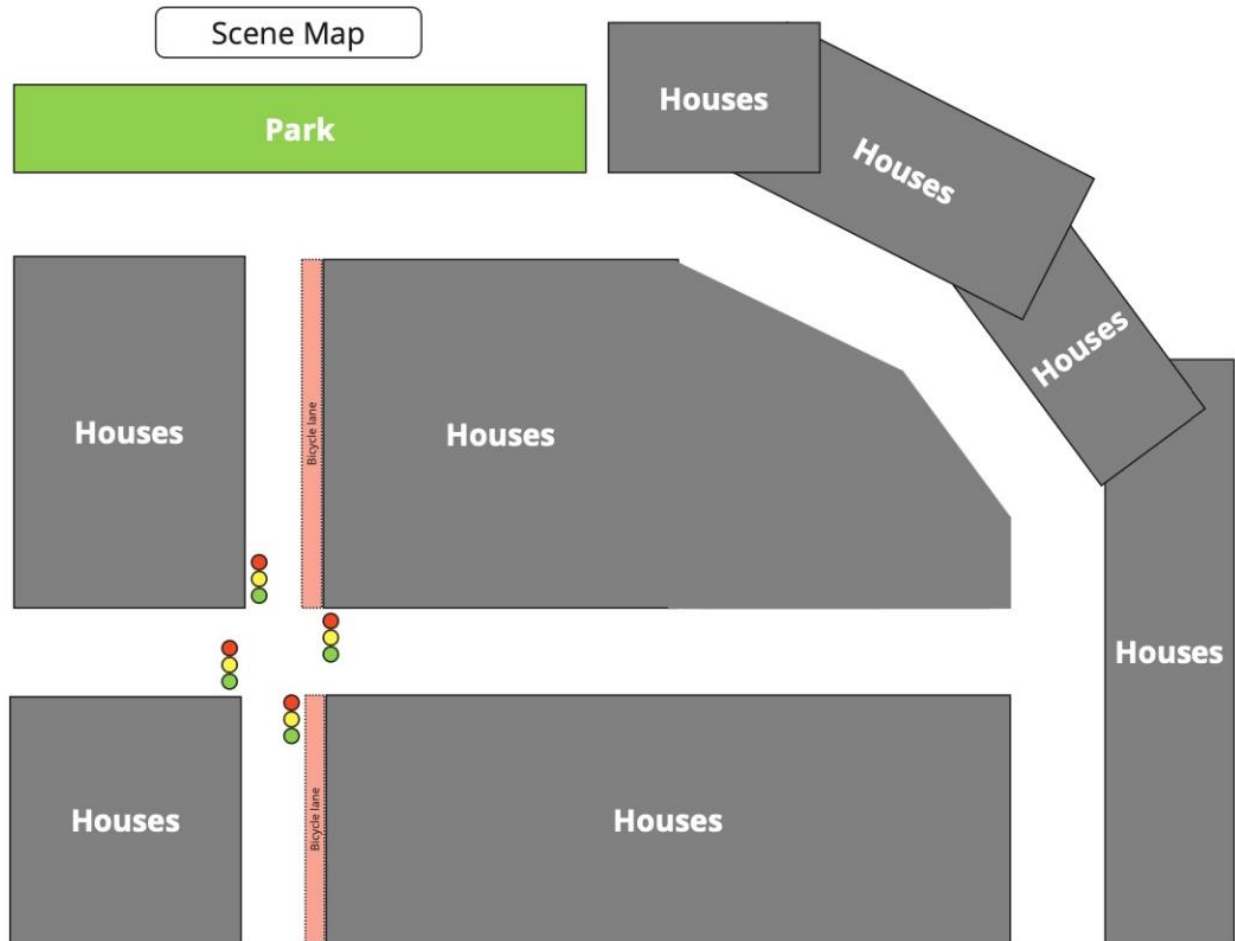


Figure 2: Map of the Environment

The following Unity Packages were used to build the environment:

Table 1: Assets used in the Environment Setup

Package	Creator	Houses	Streets	Traffic Regulation	Cars	Decoration
German City Pack 01	Toxtronyx	x	x			x
Russian buildings pack	MadMedicSoft	x				
Old Hangar	PULSAR BYTES	x				
Modular Period Building No 1, Munich, 1900	Sybaris	x				
UK Terraced Houses Pack FREE	rik4000	x				
Parking Garage - Complete	255 pixel studios	x				
Traffic signs Textures - Free Edition	VIS Games			x		
City Traffic Lights Pack [FREE] - Low Poly 3D Art	Tarbo Studios			x		
ARCADE: FREE Racing Car	RaioBoss3D's Models				x	
Down Town Pack - Lite	Gamemag Creation Studio					x
Speed Trap PBR	VIS Games					x
RPG/FPS Game Assets for PC/Mobile (Industrial Set v2.0)	Dmitrii Kutsenko					x

This is what the combination of these assets looks like in a scene, in this case Scene 1 with regular daylight:



Figure 3: Screenshot of the Environment in Scene 1

Scene 2 has the parameter for the daylight set to “sunset”, increasing the difficulty for the driver. This is what the scene looks like:



Figure 4: Screenshot of the Environment in Scene 2

5. Use Case Analysis and Scenario Derivation

This chapter describes the analysis performed on the use case document provided by PendlerRatD, an initiative to support the regular usage of bicycles. PendlerRatD specified use cases clustered in five chapters:

1. Use cases ONLY by bike
2. Use cases bicycle and car
3. Use cases bicycle and pedestrian
4. Use cases bicycle and truck/bus
5. Use cases with varying environments

For this project, only chapters 1 and 2 were declared as relevant. These chapters contain the following use cases:

R1 - Removing Barriers and building security

R2 - Expansion of driving practice (mainly on intersections)

R3 - Expansion of driving practice (mainly overtaking and narrow spots)

R4 - Expansion of driving practice at different times of the day

R5 - Weather influences when cycling

A1 - Building confidence on the bicycle path next to the road

A2 - Building confidence when driving on the road

A3.1 - Feeling safe when being overtaken by cars

A3.2 - Measuring stress when being overtaken by cars

A4 - Cycling at night and sharing the road with other cars

A5 - Cycling under varying weather conditions at varying times of day and at night

These use cases each contain several points. As an example, the breakdown of use case R1 is displayed in the following table:

Table 2: Breakdown for Use Case R1

Use Case Number	Description	Scenario	Direto Trainer	Rocker Plate	Brake	Rizer
R1a	Getting on the bike	1		x		
R1b	Starting to ride the bike straight on even ground, no vehicles driving past	1	x			
R1b	Raising the right hand to indicate an upcoming turn to the right	1	x	x		
R1b	Turning right —> right turns have no danger of being hit by a vehicle overtaking the cyclist, hence this comes first	1	x	x		
R1c	Raising the left hand to indicate an upcoming turn to the right	2	x	x		
R1c	Turning left	2	x	x		
R1d	Driving uphill	5	x			x
R1d	Driving downhill	5	x			x
R1e	Riding around obstacle on the floor	3	x	x		
R1f	Hitting the brakes hard because of a dangerous situation	3			x	

This procedure was applied to all eleven use cases from chapters 1 and 2. The third column in table x contains the scenario. This scenario is a custom component created specifically

for this project. It combines several use cases into an immersive virtual representation of the real world. These scenarios are described below:

Table 3: Scenario Definition

Number	Name	Description
1	Easy ride in the city	• Good weather • Bicycle lane is free • Cars keep their distance • Cars don't drive too fast • Intersection with a red light that the cyclist crosses straight • Stop sign at intersection with traffic lights • During the day
2	Difficult ride in the city	• Good weather • Bicycle lane is free • Cars keep their distance • Cars don't drive too fast • Intersection with a red light that the cyclist crosses straight • Stop sign at intersection with traffic lights • During the day
3	Dangerous ride in the city	• Obstacles on the ground, narrow points • Slow cyclists • Cars drive fast and way too close • Bad parking • There is a „Vorfahrt“ intersection, but a car takes the right of way
4	Foggy night ride in the city	• Nighttime with lantern lights • Cars pass by with headlights on • Fog makes vision difficult
5	Adventure ride in the forest	• Hilly terrain • Fog • Forest

6. Project Structure in Unity

The Project assets are structured as follows:

- Buildings: all asset packages related to buildings used in the scene
- Decoration: packages containing benches, trees and other small static items
- Functional: Render Pipeline assets, UDP configuration
- Prefabs: Bike Lane and car prefabs used several times
- Road Users: Bicycle Physics and car assets
- Roads: Packages related to roads, not all of them are used
- Scenes: User defined scenarios packaged in Unity scenes
- Scripts: Different Scripts to control cars and waypoints
- StreamingAssets: Essential SteamVR package
- XR: Auto generated OpenXR Plugin folder

The components in the hierarchy are structured after road elements (intersection, curve etc.). All assets in one corner of the intersection are grouped under one Unity object, for example: “FourWayIntersection” > “Section Left and Behind”.

7. Scenario Implementation

This chapter explains how the scenarios from the previous chapter were implemented. Instead of going into the details of each scenario, the different parameters are described.

Parameter 1: Lighting

Lighting in the Bicycle Simulator is essential for simulating different times of day and varying weather conditions. Proper lighting setup enhances realism and visibility, contributing to an immersive experience.

The DaylightController script manages the lighting conditions in the simulator. It adjusts the sun's position, intensity, and color to simulate different times of day, particularly focusing on sunset.

Key features of the implementation include:

- **Sun Light:** Represents the directional light source (sun).
- **Day and Sunset Timing:** The script sets a fixed time for the during the day and sunset to adjust the environment lighting accordingly.
- **Sunlight Rotation and Intensity:** The sun's position is adjusted to simulate the low angle of the sunset, and its intensity is reduced to mimic the softer light of the evening.
- **Ambient Light:** The ambient light color is changed to a warm orange hue to create a realistic sunset atmosphere.
- **Light Color Adjustment:** The sunlight color is modified to a soft, warm orange to enhance the sunset effect.

Parameter 2: Other Road Users

Currently, our five implemented scenarios feature two types of road users: cars and non-computer-player cyclists.

Other Road Users: Cars

The use case document requires the usage of cars for all use cases in chapter 2. This was achieved with the help of a Car Controller and a Car Driver AI that were mainly developed for a different project. The Car Controller defines the physics of the car including the movement of the wheels, whereas the Car Driver AI has the following components:

- Waypoint Navigation: Adjusting direction and speed to get to the next waypoint.
- Collision Avoidance: Detecting objects, bicycles and cars in the close vicinity.

These components allow the cars to interact with other elements in the hierarchy. This was implemented via raycasts detecting other objects and results in the following behavior:

- Driving cars slow down once they detect a bicycle in their vicinity.
- In case there are traffic lights, cars stop for red lights.
- In case there is a stop sign, cars stop and check the intersection.
- If the road is free, the car drives at a speed defined via the waypoint net.

All cars are bound to a predefined waypoint net. This waypoint net was developed in another project. It allows for the definition of speed per waypoint as well as a “stop” option that can be regulated dynamically via a traffic light. Furthermore, a flag indicating a left or right turn can be used to define the behavior at an intersection with stop signs.

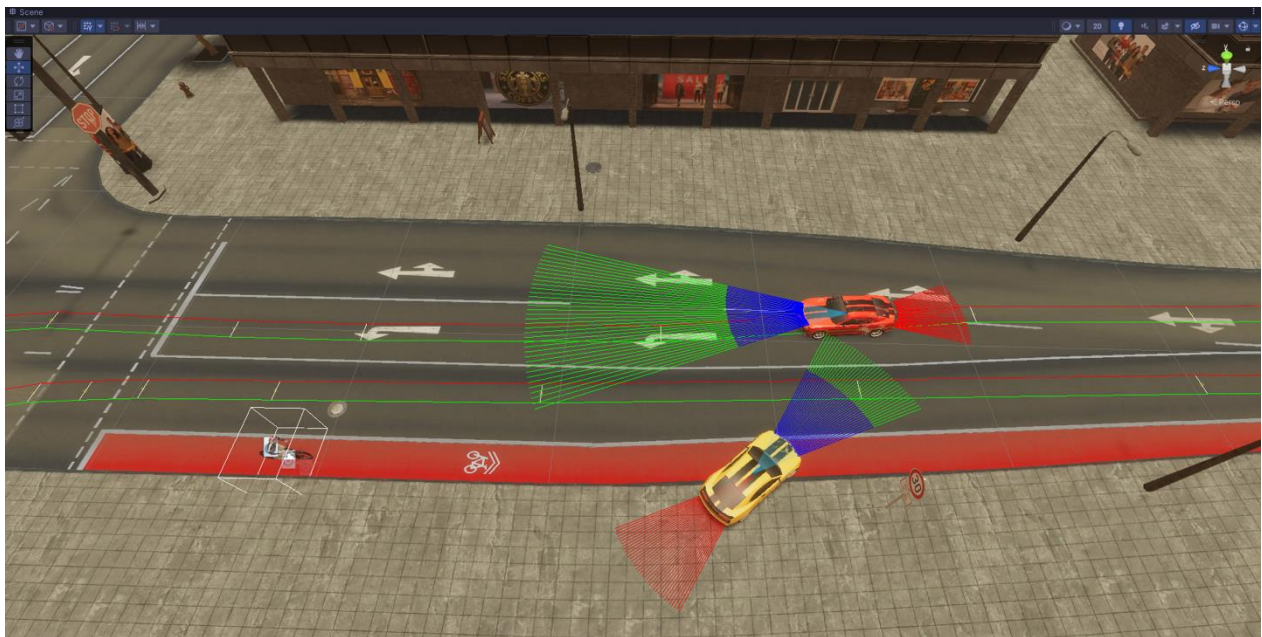


Figure 5: Scene View of two Cars interacting via Raycasts

Other Road Users: Cyclist

To create a realistic simulated environment, one of the defined scenarios includes a non-player character (NPC) cyclist who rides slowly and must be overtaken at some point. This NPC cyclist was implemented using a waypoint bicycle from the "Simple Bicycle Physics" library.

Given the extensive modifications made to the main script of this package for the player-controlled bicycle, it was necessary to add a second, unmodified version of the package to Unity. The Version Control tool in Unity was utilized to obtain this unmodified version. Additional modifications were then required to ensure both versions could run concurrently without interference.

The NPC cyclist is controlled by a waypoint system, which follows a predefined sequence of points in the simulation that the bicycle navigates through sequentially. These waypoints, along with parameters like speed, can be automatically recorded by setting the bicycle controller script to "record." This functionality is available in a separate scene called WayPointBicycle, which is included by default in the "Simple Bicycle Physics" (NPC) package. To ensure that the coordinates of the waypoints align with the bicycle lane section the cyclist should be driving on, the entire section was copied into the WayPointBicycle scene.

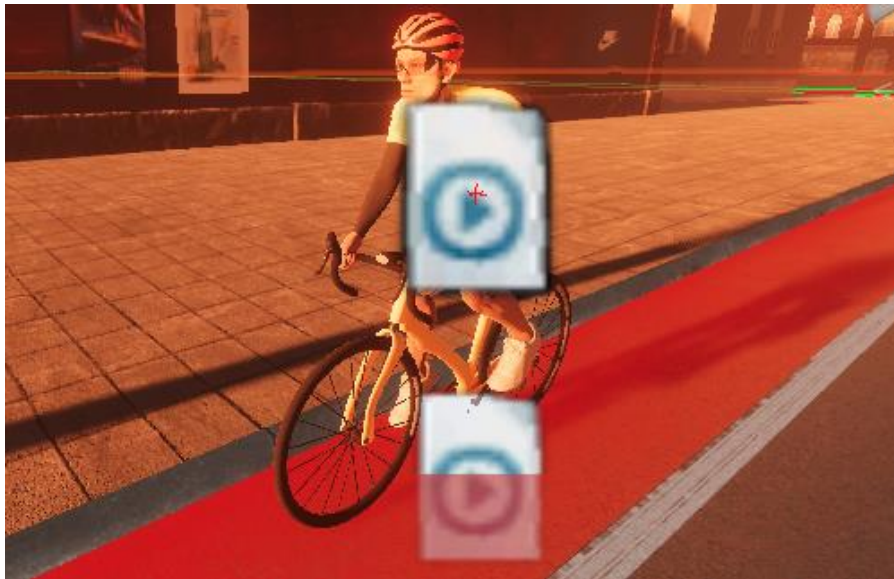


Figure 6: NPC Cyclist in the Unity Scene

While recording the waypoint system, the bike can be controlled using the WASD keys on the keyboard. Once the recording is complete, it can be saved via the menu option "Window --> Save Bicycle Replay" and later loaded for playback using "Window --> Load Bicycle Replay." It is important to activate V-sync; otherwise, the cyclist's recorded speed and playback speed will not match.

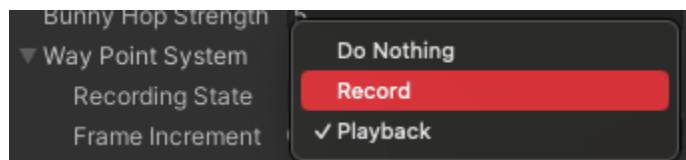


Figure 7: Way Point System Controller of NPC Cyclist

Parameter 3: Traffic Regulation

The traffic is regulated as typically seen on German Streets through the different types of traffic regulation installations:

- Pavement markings
- Traffic signs
- Traffic lights
- Different types of Road Closures

Pavement Markings

The pavement markings include:

- Lane Markings: Solid and dashed lines that indicate the boundaries of lanes.
- Directional Arrows: Arrows that show the direction of travel for specific lanes.
- Stop Lines: Lines indicating where road users should stop at intersections or traffic lights.
- Bike Lanes: Red lanes on the side of the road, supplemented with bike pictograms to enhance visibility.
- Pedestrian Crossing Lines: Dashed lines indicating the walking path for pedestrians crossing the road at intersections.

These markings are designed to create a realistic and navigable environment for both cyclists and other road users within the simulation.



Figure 8: Example of Pavement Markings (including Bicycle Lane) in Unity Scene

Further, crosswalks are also implemented with pavement markings. Additionally, a sign for each direction signaling an upcoming pedestrian crossing is added.



Figure 9: Pedestrian Crossing with Signs in Unity Scene

Traffic Signs

Traffic signs are either mounted on existing lanterns and traffic light posts, or they have their own dedicated posts. The signs themselves are created using two quads or a nearly zero-thickness cube, which are 3D Unity GameObjects. This design ensures that the signs are both realistic in appearance and functional within the simulation environment.



Figure 10: Front and Back PNG used for Sign Material

One of these quads is mapped with a material created from the front PNG image, while the other is mapped with a material created from the back PNG image. These objects are positioned very closely together to create the appearance of a single sign object with distinct

front and back surfaces. This technique ensures that the signs look realistic from both sides within the simulation.

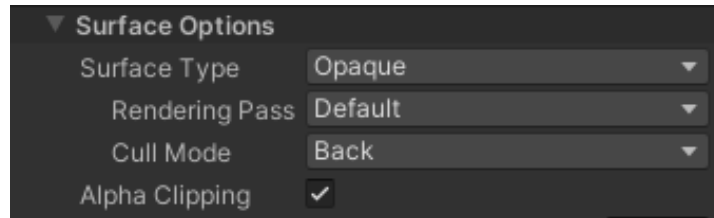


Figure 11: Surface Options of Sign Material in Unity Editor

It is important to enable alpha clipping so that the transparent part of the PNG will not be rendered.



Figure 12: Examples of different Traffic Signs in Unity Scenes

Many different types of signs can be found in the scenes for example there are speed limit signs, right of way signs, stop signs, signs signaling pedestrian crossings and much more.

Traffic Lights

Traffic lights are controlled via a controller script that allows each traffic light to be set to a specific phase. This phase determines the timing for when the traffic light will turn green. Additionally, the total number of phases required for a crossing can be defined; for example, a four-way crossing needs at least four different phases. The script also sets the emission as

the light source for the three different colors of the traffic light, ensuring accurate and realistic light transitions for red, yellow, and green phases.



Figure 13: Example of the three Traffic Light Phases in a Unity Scene

8. Integration with Elite Rizer

This chapter describes how the Elite Rizer was integrated into the setup of the bicycle simulator. After introducing Bluetooth Low Energy (BLE) as the core technology, the Process of Reverse Engineering the Elite Rizer is outlined. As a central achievement of this project, this guideline can be used to gain control the Rizer via BLE.

Bluetooth Low Energy

Bluetooth Low Energy (BLE) is designed for low power consumption, ideal for IoT devices. BLE uses services and characteristics, identified by UUIDs, to define and interact with device functionalities. Services are collections of characteristics, each representing a data point or feature with operations like read, write, and notify. Devices advertise their capabilities, establish connections, and perform read/write operations on characteristics. UUIDs ensure unique identification, with standard and custom options available for services and characteristics.

Reverse Engineering Rizer

Our first approach was to look in the nRF connect app and look at the attributes of the target devices, but that was not very helpful. The second idea was to use the nRF sniffer dongle with Wireshark on the Computer and use a Smartphone with the vendor app and look if there are some interesting packages. But we just saw the advertising packages, so that wasn't very helpful either. Presumably because of the BLE security features, we can't get useful packages.

The Book *Develop your own Bluetooth-Low-Energy Applications* [2] helped to get a better working approach. The Idea was to use the Android App from Elite and sniff the package for the important commands. In our case, '+' and '-' are used to increase or decrease the incline of the RIZER. To Sniff the BLE traffic, we used the HCI snoop log direct on our Android device. In the developer options on the Phone, it is possible to enable 'Bluetooth HCI snoop log' as well as USB debugging. To evaluate the logs, we used Wireshark on a Computer. To get the logs from the Smartphone to the computer, we need first to install the Android Debug Bridge (adb). After this, we can connect the phone over USB-C to the computer and pull the logs with these two commands:

```
adb Device
adb pull /sdcard/btsnoop_hci.log
```

Figure 14: Installation Command for Android Debug Bridge

The first command is to establish a debug connection. The phone will ask you if you allow the connection. The secondary command pulls the log from the smartphone and saves it on your computer in the current directory. In Wireshark can we open now the “btsnoop_hci.log” file. With the display filter “btatt.opcode == 0x12” can we see all write requests on the Bluetooth Attribute Protocol (ATT) the Phone has done since we started the log recording. And we can also see the payload for each service. With the help of this the data in this payload we could determine that “060102” is the payload for up and “060402” stands for down.

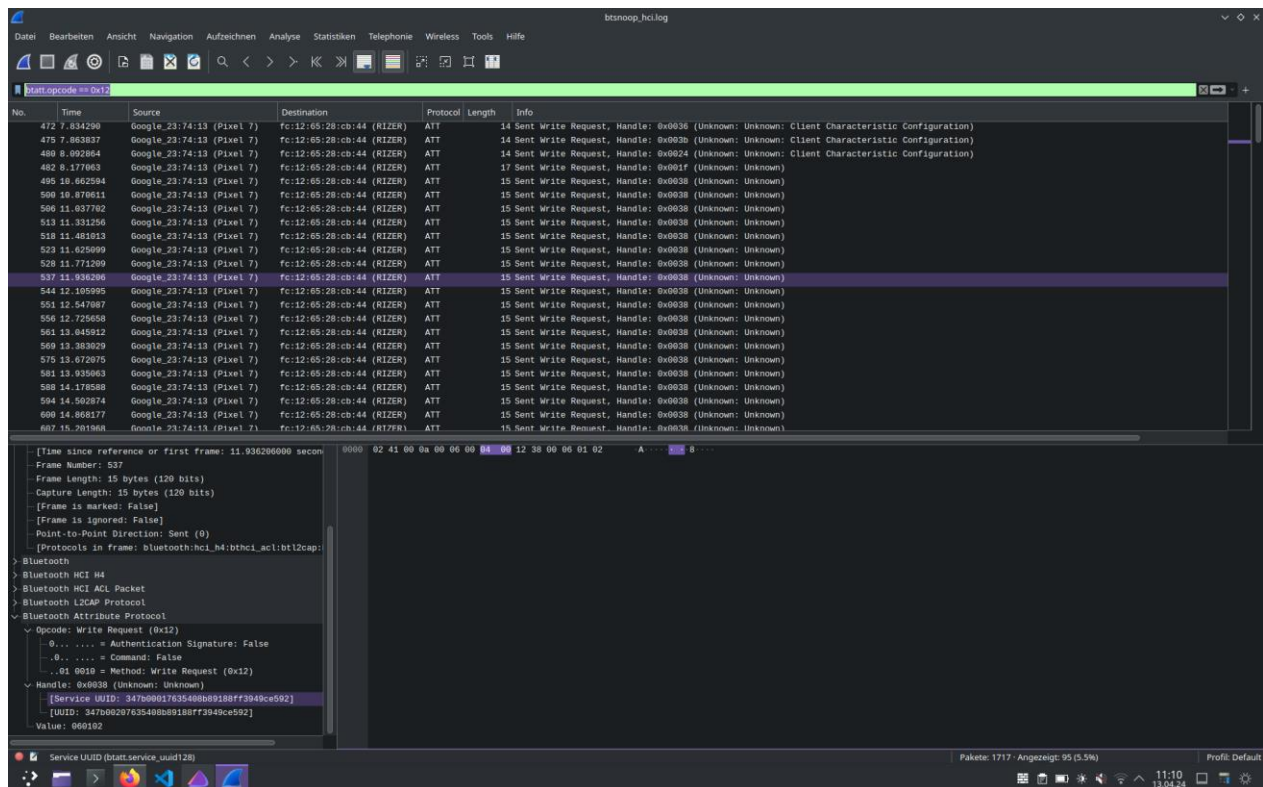


Figure 15: Bluetooth Connection Logs

Using the relevant Bluetooth commands for the RIZER, we customized the existing Rizer Python script to add write functions alongside the read function. The main challenge was needing a UDP receiver to get data over UDP from Unity and run all three tasks (sending Bluetooth, receiving Bluetooth, and receiving UDP) simultaneously and in real time. Therefore, we used the *asyncio* python library. We attempt to make for each task an own async task, but it made debugging much more difficult, and we discarded this approach.

The basic plan also provided for that we have one master collector file with a Data Receiver class who gets the data from unity Data Sender class that receives all data from the sensors and sends it bundled to unity. Every script should now just read the class variable from the

Data Receiver to get the data from unity. But there was the problem that just one script could read the right data from the class. All the other files got wrong data. We devised a workaround and incorporated a listener into each script that required data from Unity to write it on a BLE device. At the conclusion, the data receiver obtained all the pertinent information in a single JSON file from Unity and transmitted a separate JSON file for each script, containing all the necessary data.

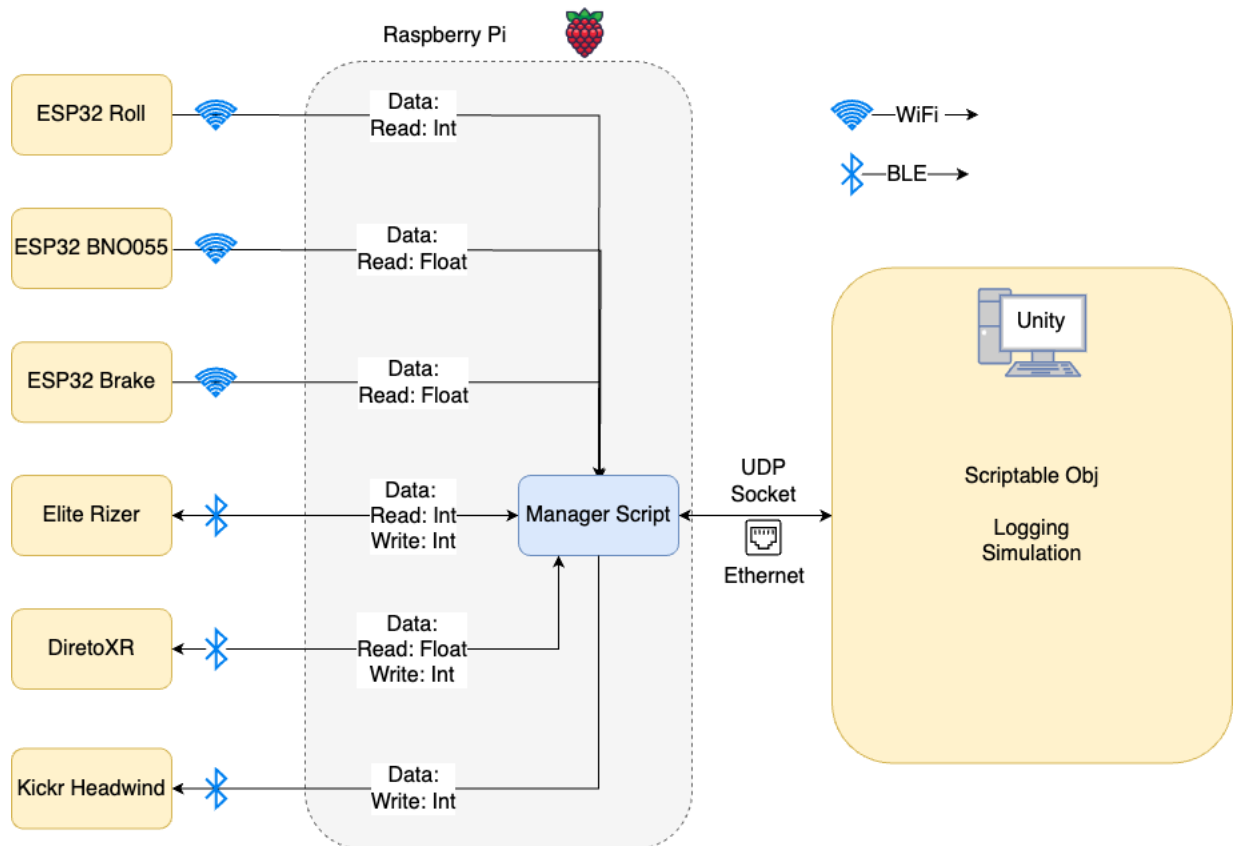


Figure 16: Sensors & Actuators Architecture

9. Summary and Next Steps

This semester, significant progress has been made on the setup of the bicycle simulator and the simulation in Unity. One major enhancement in the bicycle's hardware is the successful implementation of automatic adjustment for the Elite Rizer based on Unity input. Additionally, five new scenarios have been developed in Unity, each uniquely designed to cover a wide range of use cases which have been requested by PendlerRatD. These implementations now enable testing the functionality and realism of the bicycle simulator in various environments.

The work on this bicycle simulator will be continued by two of the four group members as part of a research project. This project will focus on further enhancing and fine-tuning the sensors and actuators, as well as improving the communication to and from the virtual Unity scene. Additionally, the research project will include a study evaluating the quality and applicability of this simulator for further research purposes.

10. Sources

- 1) Cyclophobia (Fear of Bicycles) - Psych Times. (2021, September 13). Psych Times.
<https://psychtimes.com/cyclophobia-fear-of-bicycles/>. Accessed 01.07.2024.
- 2) Vervloesem, K. (2022, May). *Develop your own Bluetooth Low Energy Applications for Raspberry Pi, ESP32 and nRF52 with Python, Arduino and Zephyr*. Elektor International Media B.V.